

GET-SYBERS / DX_DFIR

CAR Cross-Source Linkage & Detection

A Research Journey

*A timeline based on **behaviour rather than artefacts** — cascade many sources into CAR objects, wind a deterministic STIX-minted **spindle identity** onto every row, converge the same entity across sources, and let threat-intel detections light up behaviour over that cascaded data.*

docs/research · the CAR cross-source linkage & detection arc

An introduction and nine steps

Compiled 12 September 2026

Contents

Introduction — the research journey & map

01 Volume GUID as a first-class join key (B1)

02 MAC address from v1 GUIDs (B3)

03 Network identity: DNS/SSL/x509 → resolved flows (B2)

04 Preserving REG_BINARY bytes in the plaso lane (B4)

05 A YARA ruleset for unnormalised values (B5)

06 The behaviour timeline (and psort vs CAR)

07 Wiring the detection lanes into the collection

08 Detection ↔ CAR correlation

09 Behaviour sightings (detections as Sightings of ATT&CK over the spindle)

CAR cross-source linkage & detection — research journey

This directory records, one doc per step, the arc of work that turned the PIIAT-MitreCar engine from “*emits CAR objects*” into “*resolves entities across sources and lines threat detections up against them.*” It is the narrative companion to the raw findings in `../car-provenance/` (the per-object provenance catalogues and the `crosslink/` value hunt).

The north star

A timeline based on **behaviour rather than artefacts** — cascade many sources into CAR objects, wind a deterministic STIX-minted **spindle identity** onto every row, converge the same entity across sources, and let threat-intel detections light up behaviour over that cascaded data.

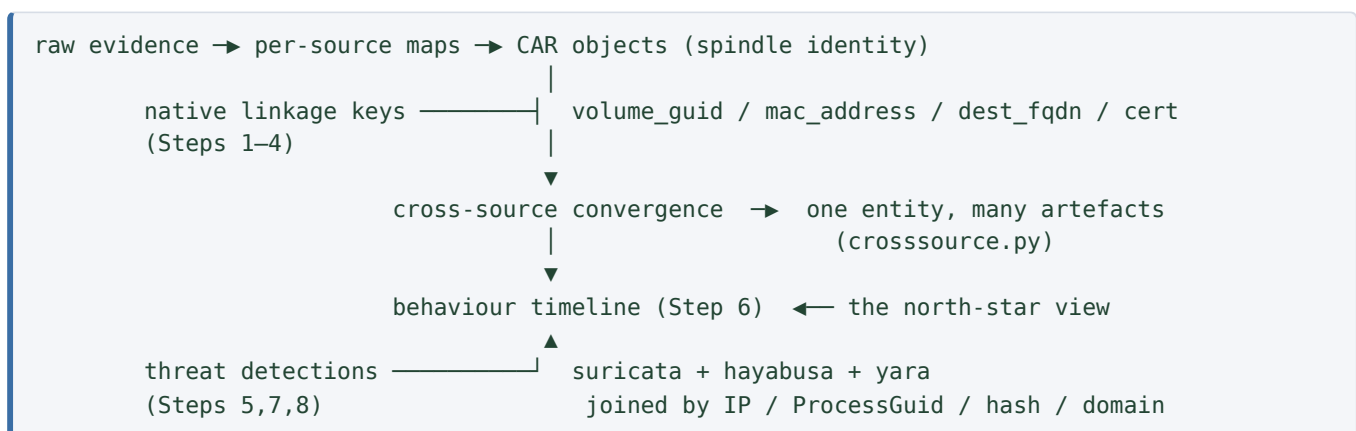
Two problems stood in the way, and the value hunt named them precisely:

1. **The join key was synthetic-only.** Of 44,327 non-null values across the CAR `guid / owning_guid / parent_guid / target_guid` columns, **zero** were canonical `{8-4-4-4-12}` GUIDs — the real cross-source keys (volume GUID, MAC, MachineGuid, domain ↔ IP) survived only as text inside `native`. Steps 1–3 lift them into first-class, converge-able keys.
2. **Detection was hollow.** The detection lane produced nothing but hayabusa on a sorted collection — the lanes weren’t wired to the collection and had no rulesets. Step 7 fixes the pipeline; Step 8 is the payoff: real IDS/EDR detections joined to the resolved CAR data.

The steps

#	Step	What it delivers
1	Volume GUID as a first-class join key	the strongest disk key (6 data_types) mined from native + a queryable column
2	MAC address from v1 GUIDs	the NIC MAC decoded out of DLT/volume GUID nodes → a device-linkage key
3	Network identity: DNS/SSL/x509 → resolved flows	a bare C2 IP reads as its domain + certificate (the C2 chain, resolved)
4	Preserving REG_BINARY bytes in the plaso lane	device serials/MACs survive extraction instead of collapsing to (N bytes)
5	A YARA ruleset for unnormalised values	flags distinctive values the pipeline hasn't yet normalised
6	The behaviour timeline (and psort vs CAR)	2.7M-event behaviour timeline over the full LoneWolf disk; the layer distinction
7	Wiring the detection lanes into the collection	<code>process all --fetch</code> runs suricata/yara/hayabusa on the collection's evidence
8	Detection ↔ CAR correlation	suricata + hayabusa alerts joined to the resolved/normalized CAR entities
9	Behaviour sightings	the join emitted — detections as STIX Sightings of ATT&CK attack-patterns over the spindle observed-data

How the pieces compose



Provenance & scope caveat

The processed corpus is **multi-host** — three collections, different machines:

- **lonewolf** — the LoneWolf disk image (`DESKTOP-PM6C56D`); disk only. Source of the full CAR `car.db` / behaviour timeline (Step 6).
- **ls24-sample** — pcaps (incl. the DFIRdump C2 capture), a memory image, a VMDK, and a Sysmon `log.evtx` (`DESKTOP-M913391`); the network + evtx detection story (Steps 3, 8).

- **2019-narco** — a further disk collection.

Cross-source convergence is naturally **host-scoped**: it lights up *within* a host, so the richest single-host line-up is the `lonewolf` disk (a follow-up), while `ls24-sample` carries the network + host detections used in Step 8.

PR ledger

- **Engine (PIIAT-MitreCar)**: #57/#58/#59 volume GUID, #60 MAC, #61 DNS + #65 SSL, #66 x509, #63/#65 timeliner robustness, #54 behaviour layer (prior).
- **DX_DFIR**: #153 plaso byte-preservation, #152/#154 YARA ruleset, #159/#161/#162 detection-lane wiring, #163 this research journey, the behaviour-sightings bridge (`stix/behaviour.py`) + the yara memory-lane renderer fix, plus the submodule pin bumps that carried each engine change onto `main` .

Step 01 — Volume GUID as a first-class join key (B1)

Part of the CAR cross-source linkage & detection research arc — see [README](#) for the full map.

Status: merged (PIIAT-MitreCar#57, PIIAT-MitreCar#58/#59)

The gap

CAR’s `guid` is the one cross-source join key: it travels to Elastic as `ECS event.id` on every object and `process.entity_id` on process rows, and `LOOKUP JOIN car-detections ON event.id|process.entity_id` is the entire convergence contract (`python/get_sybers_dxdfir/detect/rules/car-detections/join-keys.yml`).

The cross-dataset value hunt (`../car-provenance/crosslink/`) found that key holds **nothing real**. Of **44,327** non-null values across the CAR `guid` / `owning_guid` / `parent_guid` / `target_guid` columns, **zero** are canonical `{8-4-4-4-12}` GUIDs — every value is a synthetic id the engine mints itself (`proc-...`, `file-...`). So no MachineGuid, no volume GUID, no Sysmon ProcessGuid is ever promoted to a queryable field; the real cross-source keys survive only as text inside `native`. The single strongest such key on a disk image is the volume GUID (`\\?\Volume{GUID}`), and it was entirely un-mined.

What we found

One volume id — `{09931f21-7faf-44a9-81d8-1e73c14b9eaf}`, the LoneWolf main system volume, seen **2,208×** — ties **6 data_types** together on real data (see `crosslink/guids.md`):

data_type	rows
<code>windows:evtx:record</code> (event log)	1082
<code>windows:registry:key_value</code>	34
<code>fs:ntfs:usn_change</code> (USN change journal)	14
<code>fs:stat</code>	12
<code>google_drive_sync_log:entry</code> (cloud-sync)	5
<code>windows:registry:mount_points2</code>	2

One `LOOKUP ... ON volume_id` fuses disk activity, the Windows logs, the mount table and cloud exfil-sync for the case — but no CAR field carried it. The catch: the value appears mixed-case in the wild (`09931F21...` / `09931f21...`), and a naive GUID grab would drown in the ubiquitous COM CLSID / interface-IID GUIDs (e.g. `f750e6c3-...` at **118k×** — pure linkage noise present on every Windows box).

The fix

PR PIIAT-MitreCar#57 added a `definitive_native_id` convergence tier in `crosssource.py` that mines `Volume{GUID}` out of `native`. It is token-gated so the COM CLSID/interface GUID families are never picked up, case-folded so the real mixed-case values collapse to one key, and host- and object-independent.

PR PIIAT-MitreCar#58/#59 promoted `volume_guid` to a first-class **nullable header column** on every CAR object: - a shared extractor lives in `native_ids.py`; - `enrich` lifts it **fill-only-null** (never overwrites an existing value); - the CAR model snapshot was regenerated to include the new header; - it projects to a custom `car.volume_guid` field — ECS 8.11 has no volume-GUID field (verified), so the cross-object key homes in the custom namespace.

What it enables

The volume GUID is now both a **queryable column** and a **cross-source join key**. A registry mount entry, a USN change-journal record and an event-log row that name the same volume converge on one key — the best bridge available on a disk image, previously thrown away.

Follow-ups

The rest of B1 remains open (tracked in `crosslink/SUMMARY.md`): - lift **MachineGuid** → `host.id` and stamp it on every CAR row of the image; - in an evtx → CAR projection, map Sysmon **ProcessGuid** → `process.entity_id` and **ParentProcessGuid** → `parent_guid`; - populate **owning_guid** on registry and file rows — currently NULL on all 12,171 registry and 31,828 file rows, so the synthetic tree is still incomplete.

The sibling MAC key from the same value hunt is covered in [Step 02](#).

Step 02 — MAC address from v1 GUIDs (B3)

Part of the CAR cross-source linkage & detection research arc — see [README](#) for the full map.

Status: merged (PIIAT-MitreCar#60)

The gap

A version-1 GUID (the RFC 4122 time+MAC variant) carries the originating NIC's MAC address verbatim in its node bytes. The cross-dataset value hunt (`../car-provenance/crosslink/`) counted **~6,218** MAC-bearing v1 GUIDs in the LoneWolf image alone: the DLT birth-droid stamped into every LNK shortcut, and the `-11e2-` / `-11e8-` volume and network- interface GUIDs. On top of those sit literal MAC strings, such as the NetworkList gateway MAC.

None of that was ever lifted into a CAR field — no `mac_address` column existed anywhere in the model. So a file opened through a shortcut could not be tied back to the machine that created it: the attribution key was sitting inside the GUID and being thrown away.

What we found

The node decodes cleanly to real hardware MACs (see `crosslink/guids.md`):

- `5c2307d9-3369-11e2-be70-001cc42df40b` → `00:1c:c4:2d:f4:0b` — the DLT birth-droid shared by 96 `windows:lnk:link` rows and 32 `windows:distributed_link_tracking:creation` rows: which volume/machine a shortcut's target was born on.
- `{3869c27a-31b8-11e8-9b12-ecf4bb487fed}` → `EC:F4:BB:48:7F:ED` — a real Dell MAC leaked by the MountPoints2 volume GUID.

Not every v1-shaped node is a hardware MAC, so three families are excluded: - the OLE/COM node `0000000000046` (the `...-c000-0000000000046` family); - the RFC synthetic placeholder `806e6f6e6963` (e.g. `{5c3108bb-31c0-11e8-9b10-806e6f6e6963}` — an `80:6e:6f:6e:69:63` non-address); - any node whose first octet has the multicast / I-G bit set — an RFC-random, non-hardware id rather than a burned-in NIC address.

The fix

PR PIIAT-MitreCar#60 added `native_ids.mac_from_v1_guid`, which decodes a v1 GUID's node into a hardware MAC and applies the three exclusions above. `native_ids.mac_addresses` additionally gathers literal `xx:xx:xx:xx:xx:xx` strings out of `native`.

`mac_address` became the **3rd non-MITRE header column** (after `owning_guid` and the `volume_guid` from [Step 01](#)): - `enrich` lifts it onto every object; - `crosssource` converges rows on a `mac` class; - it projects to a custom `car.mac_address` field — ECS's MAC fields are role-specific (`source.mac` / `destination.mac` / `host.mac`), so a cross-object header key belongs in the custom namespace, not any one role.

What it enables

A LNK's DLT birth-droid and a NetworkList entry that share a NIC MAC now join on one key — device-linkage the pipeline entirely lacked. The MAC becomes both a queryable column and a convergence class, alongside the volume GUID from [Step 01](#).

Follow-ups

Two device-linkage gaps from the same hunt remain (see `crosslink/SUMMARY.md`): - **B4 · plaso data loss** — `MountedDevices` / `DefaultGatewayMac` / device bytes are emitted only as `(6 bytes)` / `(224 bytes)` summaries, so the raw MAC/serial is dropped by the parser before any normalisation could recover it (a `lane/parser-config` fix, upstream of this decoder). - a **device-serial** CAR field still does not exist, though USB iSerials and USBSTOR/MountedDevices bind USB devices to volume GUIDs (e.g. the SanDisk Extreme serials seen 2,470× across the disk host).

Step 03 — Network identity: DNS/SSL/x509 → resolved flows (B2)

Part of the CAR cross-source linkage & detection research arc — see [README](#) for the full map.

Status: merged (PR PIIAT-MitreCar#61, PR PIIAT-MitreCar#65, PR PIIAT-MitreCar#66)

The gap

Zeek’s network logs are the richest single source of flow evidence in the pipeline, but a triage of the processed tree showed the Zeek family looking “entirely raw” — no CAR objects where we expected flows. The instinct was that the mappings were missing. They were not: `conn → flow`, `http → http`, `smtp → email`, and `files → file` were already in place, roughly ~80% of the family mapped. The blank tree was a **lane-execution gap** — the maps existed but were not being run — not a modelling gap.

Once the lane ran, the *real* remaining gap surfaced: three logs carried no CAR mapping at all — **dns**, **ssl**, and **x509**. These are exactly the logs that carry network *identity*. Without them, an encrypted connection to a bare IP address stayed a bare IP: no domain, no certificate, nothing to correlate the value hunt’s C2 chain against.

What we found

Each of the three logs identifies its records differently, and getting the identity form right was the crux of each fix:

- **dns** — one UDP:53 connection carries *many* queries, so `uid` alone is not unique. Records are keyed by `uid` + `trans_id`. This needed a new spindle form, `zeek_uid_trans_id`.
- **ssl** — `ssl.log` is **1:1 with the connection**, unlike dns and http, so `uid` alone is a stable identity (`guid = uid`).
- **x509** — the Zeek `fingerprint` field is a SHA-256 of the DER certificate. That means it maps straight onto `sha256_hash` — a free content-hash convergence with the file model, no re-hashing.

The fix

DNS (PR PIIAT-MitreCar#61). `zeek_dns → flow` (message), the queried name in `fqdn`, raw answers kept in native. `enrich._dns_resolution` builds a **host-scoped ip → resolved domain map** from each (query, answer-IPs) pair, and `_stamp_resolved_fqdn` fills `dest_fqdn` / `src_fqdn` on any flow to or from a resolved IP. So a bare connection to `100.101.0.42` now reads as **scoring-c2.berylia.org** — the C2 chain the value hunt found, now queryable. Only IP answers resolve; CNAMEs are skipped. Resolution is gated on `source_artefact == "zeek_dns"`, because a `conn` flow with `service:dns` is DNS *traffic*, not DNS *resolution evidence*.

SSL/SNI (staged in PR PIIAT-MitreCar#65). `zeek_ssl → flow` (message); `server_name` (the SNI) → `dest_fqdn`; `guid = uid`. `enrich` propagates the SNI onto the `conn` flow of the same `uid`, so the domain lands **directly on the encrypted flow** even when DNS never resolved it.

x509 (PR PIIAT-MitreCar#66). `zeek_x509 → file` (create); `fingerprint → sha256_hash`; `subject` / `issuer` / `SAN` / `validity` kept in native. `enrich._cert_by_fingerprint` and `_stamp_flow_cert` walk `ssl.cert_chain_fps → x509.fingerprint` and surface the **leaf cert subject** on the TLS flow that presented

it. The C2 flow now reads `CN=berylia.org` alongside its SNI, and `sni_matches_cert` is the mismatch tell when the two disagree.

What it enables

An encrypted flow to a bare IP now carries its C2 domain — from DNS resolution *and* from the SNI on the flow itself — plus the certificate identity of the endpoint it handshook with. That is the network-identity substrate the detection-correlation work ([Step 08](#)) joins against: domains, certs, and IPs are now first-class, cross-referenceable fields rather than opaque addresses.

Follow-ups

None major. `ssl` and `x509` close out the Zeek family — every Zeek log the pipeline ingests now has a CAR mapping.

Step 04 — Preserving REG_BINARY bytes in the plaso lane (B4)

Part of the CAR cross-source linkage & detection research arc — see [README](#) for the full map.

Status: merged (PR #153)

The gap

The plaso lane was silently discarding raw registry bytes. Plaso renders Windows registry **REG_BINARY** values as a size summary — a literal string like `(N bytes)` — instead of the bytes themselves. Crucially, this happens at **log2timeline** extraction time, in `plaso/parsers/winreg_plugins/interface.py`, *before* any normalization in our pipeline can see the value. By the time a record reaches CAR mapping, the bytes are already gone.

This is not a rendering choice we could undo downstream: re-running `psort` cannot bring the bytes back, because the loss is at **extraction**, not at output formatting. Whatever `log2timeline` did not store, `psort` has nothing to render.

What we found

The dropped **REG_BINARY** values are exactly the ones that carry hardware and device identity — some of the most decisive evidence for tying a volume, a drive letter, and a physical device together:

- **MountedDevices** blobs — the binding between a volume, its drive letter, and the USB serial of the device that mounted there.
- **Tcpip** device values.
- **DefaultGatewayMac** — a 6-byte gateway MAC address. (NetworkList decodes its own copy, but the `Tcpip`-side value was being lost.)

Each of these arrived as `(N bytes)` — a count, not content — leaving nothing to normalize.

The fix

PR #153 added the `--extract_winreg_binary` flag to the `log2timeline` invocation in `run_plaso` (`python/get_sybers_dxdfir/plaso.py`). With the flag set, `log2timeline` retains the raw bytes, and because the JSON output module already `base64url`-encodes `bytes` values, the bytes now survive intact through to the JSON we ingest.

Concretely, `HKLM\System\MountedDevices` value `\DosDevices\C:` went from the useless `"(12 bytes)"` to bytes that decode to:

```
c4 81 c4 81 00 7e 00 00 00 00 00 00
```

which is disk signature `0x81C481C4` followed by partition offset `32256` — the actual volume-to-device binding, now recoverable.

What it enables

The device-identity bytes now survive extraction and reach the pipeline as base64url in the JSON output, so downstream normalization finally has something to work with. The volume ↔ drive-letter ↔ USB-serial binding, the `Tcpip` device values, and the gateway MAC are present rather than summarized away.

Follow-ups

B4 only guarantees the bytes **survive extraction**. Decoding those raw bytes into normalized CAR fields — a MAC field, a device-serial field, a disk signature — is a separate downstream step still to be done. This step secures the raw material; it does not yet model it.

Step 05 — A YARA ruleset for unnormalised values (B5)

Part of the CAR cross-source linkage & detection research arc — see [README](#) for the full map.

Status: merged (PR #152, Copilot fixes in PR #154)

The gap

The cross-source value hunt ([Step 04](#) and the `docs/car-provenance/crosslink/*.md` notes) kept surfacing the same shape of problem: a distinctive value is present in the raw/native artefact but is never lifted into a CAR field, so it cannot be used as a cross-source join key.

The hunt had already produced roughly **30 YARA rule stubs**, but they were scattered one-by-one across the crosslink notes (`cmdlines.md`, `guids.md`, `host_ip.md`, `identity.md`, `ports_serials.md`). They were illustrative, not runnable — you could not point them at a processed dataset and get a report of what the pipeline had failed to normalise.

What we found

The stubs fell into a small number of recurring value classes, each keyed to a CAR field that *should* have received the value:

- **Host-identity GUIDs** — `MachineGuid`, interface GUIDs, Sysmon `ProcessGuid` lineage, scheduled-task GUIDs, imaging-tool GUIDs.
- **Volume / mount** identifiers.
- **MAC-bearing v1 GUIDs** — the node field of a time-based UUID is a real MAC.
- **Network 5-tuple** — IPv4, IPv6, MAC, hostname, ports.
- **Device serials** — e.g. `USBSTOR`.
- **Account / SID identity** — including usernames mined from `\Users\<name>\`.
- **Command-line** obfuscation, C2, and LOLBin indicators.

We also found a large volume of GUID-shaped noise that is *not* worth a rule: COM CLSID / interface / TypeLib / AppID GUIDs, the OLE `c000-...-46` family, and the `806e6f6e6963` synthetic-MAC placeholder. These are host-invariant or constant, so they carry no linkage value.

The fix

PR #152 consolidated the scattered stubs into one reusable ruleset, `docs/car-provenance/crosslink/unnormalised-values.yar` — **33 rules**: 23 general `Gap_*` rules plus 10 case-specific `HUNT_Case_*` rules across the categories above. Every rule's `meta` block carries a `car_gap` tag naming the CAR field or object the value should feed — so a hit reads as “this value exists and belongs in *that* CAR field, but isn't there.”

The suppressed-as-noise classes are documented explicitly with **no rules** (COM CLSID/interface/TypeLib/AppID GUIDs, the OLE `c000-...-46` family, the `806e6f6e6963` synthetic-MAC placeholder), so the omission is a

decision on the record rather than an oversight.

The ruleset compiles clean against **YARA 4.5.2** (the `dxdfir/yara` image).

Copilot fixes (PR #154):

- The SSH-port detector used a bare `":22"` string, which matched IPv6 substrings and timestamps. It was replaced with an octet-validated IPv4 endpoint regex that reuses `Gap_Net_IPv4_Address`, so a “port 22” hit is now anchored to a real IPv4 address.
- Multiple string declarations were split one-per-line for readability and to avoid ambiguous grouping.

What it enables

Run the ruleset over any future processed dataset and it flags the distinctive values the pipeline has *not* normalised — directly operationalising the user’s ask to “use YARA to limit missing non-normalised data.” It turns the value hunt from a manual reading exercise into a repeatable coverage check, and each hit’s `car_gap` tag points straight at the CAR field that needs a mapping.

Follow-ups

- Feed confirmed `car_gap` hits back into the CAR mappers so recurring classes stop being gaps.
- Fold the ruleset into the detection lane once that lane is wired and carries real rulesets — see [Step 07](#).

Step 06 — The behaviour timeline (and psort vs CAR)

Part of the CAR cross-source linkage & detection research arc — see [README](#) for the full map.

Status: merged (timeliner robustness fix on engine `main`, PIIAT-MitreCar#65)

The gap

The north star for this arc is **a timeline based on behaviour rather than artefacts**. `byakugan/timeline.py` builds it from `car.db` (with `superset.db`): it walks the normalized CAR objects and the relationship edges between them and emits them time-ordered — a behaviour view, not a per-parser event dump.

Two things were unresolved: the timeline builder had never been run end-to-end at scale, and there was an open question about how it relates to plaso's `psort` super-timeline — whether the two are the same thing.

What we found

The first end-to-end run crashed. `build_timeline` iterated over *every* table in the database and assumed each carried the CAR header, so it died with `no such column: timestamp` on PIIAT-Mem's auxiliary `image_context` table — a table that legitimately has no timestamp column.

psort and the CAR timeliner are different layers — not substitutes. The question got a concrete answer: plaso's `.plaso` storage file is itself a SQLite database, and `psort` runs on it to produce the **artefact super-timeline** — one row per parser event. The DX_DFIR plaso lane already produces this. The CAR timeliner operates on a *different* schema: normalized, cross-source CAR objects plus behaviour, stored in `car.db`. `psort` cannot read `car.db` (wrong schema) and cannot read a memory image at all. The two are **complementary**: artefact super-timeline underneath, behaviour timeline on top.

The fix

The crash fix makes `build_timeline`:

- **skip any table without a `timestamp` column**, so auxiliary tables like `image_context` no longer abort the run; and
- **escape the table identifier** before interpolating it into the query.

What it enables

With the fix in place, the full **LoneWolf disk image** ran end to end:

- 4,169,774 plaso rows → 2,773,205 CAR events, in ~8 minutes, into a 4.3 GB `car.db`.
- CAR event breakdown: registry 1,511,021 · file 1,253,881 · process 3,738 · http 2,012 · authentication 1,616 · user_session 880 · service 57, plus 4,723 relationships.
- Behaviour timeline: 2,701,408 time-ordered entries.

The user-from-path enrichment (A1) proved out at scale: it mined **jcloudy** on 28,946 rows where plaso's native username was `-` on essentially everything — the enrichment paying for itself across the whole image.

Follow-ups

- A handful of artefacts carry epoch-zero / far-future timestamps that widen the raw span (**1970** → **2105**). A cheap sanity-clamp on out-of-range timestamps is worth adding so the span reflects real activity.
- The behaviour timeline is the substrate the detection work reads from — see [Step 07](#).

Step 07 — Wiring the detection lanes into the collection

Part of the CAR cross-source linkage & detection research arc — see [README](#) for the full map.

Status: merged (PR #159, PR #161, PR #162)

The gap

On a *sorted* collection, `dxdfir process all` produced **no suricata and no yara detections** — the output dirs came back empty. Only hayabusa produced alerts. For a detection pipeline that is a silent, wrong result: the operator sees “hayabusa found things” and reasonably assumes the other engines found nothing, when in fact they never looked at the right evidence and had no rules.

What we found

Root cause #1 — the `signatures` lane was absent from `collection.LANES`. `LANES` is the registry mapping each lane to the sorted-collection subdirs it reads and its Ansible input var. With no entry, `process all` neither ran the detection lane nor scoped its inputs. So the engines fell back to their raw defaults:

- suricata → the empty `data_store/raw/pcaps`,
- yara → the empty `raw/disk_images / raw/memory`,

while the collection’s actual sorted captures and images sat under `raw/collections/<name>/...` and were never read. Hayabusa “worked” only by accident: its loose-EVTX scan already walks `raw/` recursively, and its rules ship inside the image — so it alone had both inputs and rules.

Root cause #2 — no rulesets. Even once the lane was wired, suricata and yara had nothing to match against: the hardened images ship **no rules**.

The fix

All fixes landed in `DX_DFIR`:

- **PR #159** — added `Lane("signatures", ("pcaps",), ...)`, placed **last** in `LANES` so detection runs after the evidence lanes have populated the collection. Added a `--pcap-dir` CLI flag and the corresponding role var.
- **PR #161** — suricata **ET Open** ruleset provisioning on `--fetch`, mirroring the yara lane’s DetectRaptor fetch. ET Open is a rolling daily feed, so the pin is the **engine-version URL** rather than a content hash — this is the `suricata-update` trust model, not a reproducible-artefact pin.
- **PR #162** — extended the signatures collection mapping to **all** detection inputs: `("pcaps", "disk_images", "memory")`, driving yara disk/memory scans and hayabusa image-EVTX extraction. Added `--disk-dir / -memory-dir` CLI flags, and made `--yara-sources` **merge** rather than clobber the scoped dirs.

The final lane definition — `Lane("signatures", ("pcaps", "disk_images", "memory"), ...)` as the last entry in `LANES` — is the wiring these three PRs converged on.

What it enables

`dxdfir process all <collection> --fetch` now runs **suricata** + **yara** + **hayabusa** against the collection's own sorted evidence, with real rulesets (**ET Open** for suricata, **DetectRaptor** for yara). Detection is no longer silently scoped to empty raw defaults, and no engine is quietly ruleless.

This is the prerequisite for correlating detections across sources — the next step, [Step 08](#).

Follow-ups

- Confirm ET Open / DetectRaptor fetch behaviour in the air-gapped / no- `--fetch` path, and document what a run without rulesets should report.
- Fold the unnormalised-values ruleset ([Step 05](#)) into the yara lane so coverage checks ride the same wiring.

Step 08 — Detection ↔ CAR correlation

Part of the CAR cross-source linkage & detection research arc — see [README](#) for the full map.

Status: demonstrated end-to-end on `ls24-sample` (rests on [Step 03](#) and [Step 07](#), both merged).

The gap

The north star is not “run MITRE’s CAR analytics” — it is to let **threat detections light up behaviour over the cascaded, cross-source-resolved data**. A hayabusa alert is one evtx record; a suricata alert is one IP 5-tuple. On their own they say *what* fired, not *which entity* it belongs to across the rest of the evidence. The point of CAR is to **expand a detection beyond its own source** — to name the entity the alert touches, using everything the cascade resolved.

Until [Step 07](#), this couldn’t even be attempted: the detection lane was hollow (no wiring, no rulesets). With that fixed, the lane produces real alerts, and Steps 1–3 give CAR the resolved identities to join them to.

What we found

Running the detection lane on the `ls24-sample` collection with `--fetch` (the wiring of [Step 07](#)):

- **ET Open provisioned** — `suricata.rules` = **144,271 rules** fetched (the [Step 07](#) `--fetch` provisioning, live for the first time).
- **suricata: 14 alerts on the DFIRdump capture — every one on the C2 IP 100.101.0.42 :**
- 10 × ET INFO SSH-2.0-Go version string Observed in Network Traffic (the Go SSH C2 client)
- 2 × ET SCAN NETWORK Outgoing Masscan detected
- 2 × ET SCAN NETWORK Incoming Masscan detected

Suricata auto-derived the capture’s tuning too: `HOME_NET` , `HTTP_SERVERS=[100.101.0.42]` , `DNS_SERVERS=[100.95.95.4]` . - **hayabusa: 86 alerts on DESKTOP-M913391** — 46 × Sysmon EID 1 (process create) + 40 × EID 5 (terminate); the one high-severity alert is the DOSfuscated `"cmd /c"` execution (RecordID 5261, ATT&CK T1059).

The correlation

Network — suricata ↔ CAR

suricata (independent IDS)	CAR (from the same Zeek capture)
14 alerts to/from <code>100.101.0.42</code> — masscan recon + a Go SSH C2 client	the flow to <code>100.101.0.42</code> resolves to <code>dest_fqdn = scoring-c2.berylia.org</code> (DNS + SNI, Step 03) and carries the <code>berylia.org</code> certificate (x509)

Suricata flags “masscan + SSH C2 to an IP”; CAR **names that IP as the C2 domain and its certificate**. The IDS supplies the verdict, the cascade supplies the identity — the exact C2 chain the value hunt first spotted (`scoring-c2.berylia.org` → `100.101.0.42`), now corroborated by an independent detector and joined to a named entity.

Host — hayabusa ↔ CAR

Hayabusa's 86 **record-level** alerts normalize, through the evtx → CAR maps, into 85 **process objects** (a process's create + terminate fold into one identity by ProcessGuid). The high T1059 alert (RecordID 5261) is a CAR **process** row carrying its command line and — via **enrich** — its parent (**parent_guid**) and any spoke events (files/registry/network) the same ProcessGuid touched. That is the “expand beyond evtx”: one alerted record becomes the process and its behaviour graph.

What it enables

This is the north star, realised on real data: **threat detections joined to the cascaded, cross-source-resolved CAR entities**. A network alert on a bare IP reads as a named C2 domain + cert; a host alert on one evtx record reads as a process and its lineage. The same join keys generalise — a detection can be pivoted to memory, disk, and registry evidence of the same entity wherever the cascade resolved it.

It also sets up the *generative* half of the project: the behaviours these detections light up over the cascade are exactly the shapes to **generate new CAR analytics into** — codified against the broader set of mapped data sources, not just the ones MITRE's stock analytics assume.

Follow-ups

- **Same-host disk line-up on lonewolf**. `ls24-sample` is the network + evtx collection; the `lonewolf` disk (`DESKTOP-PM6C56D`) is disk-only, so its detections (hayabusa on image-extracted evtx + yara on the mounted disk) line up against the `lonewolf` CAR `car.db` ([Step 06](#)) — a single-host correlation where cross-source convergence is fully in play.
- **yara/memory detections** (Volatility `vadyarascan` over `memdump.mem` with the DetectRaptor ruleset) — the memory-side detections, not yet run.
- **Sightings projection** — emit the detection ↔ entity joins as STIX Sightings of ATT&CK attack-patterns over the spindle-identified observed-data (the behaviour timeline as the primary axis).

Step 09 — Behaviour sightings (detections as Sightings of ATT&CK over the spindle)

Part of the CAR cross-source linkage & detection research arc — see [README](#) for the full map.

Status: implemented (`python/get_sybers_dxdfir/stix/behaviour.py` , CLI `dxdfir stix behaviour-sightings`); demonstrated end-to-end on `ls24-sample` .

The gap

[Step 08](#) proved the correlation *by hand* — a suricata alert on `100.101.0.42` lines up with the CAR flow that resolves it to `scoring-c2.berylia.org` ; a hayabusa T1059 record lines up with a CAR process. But nothing *emitted* that join. The engine already projects its **own** CAR analytics as STIX Sightings of ATT&CK attack-patterns over the spindle observed-data (`byakugan/stix.py` + `analytics.py`) — but the DX detection lanes (suricata / hayabusa / yara) were never fused onto that behaviour axis. The DX exchange's `stix export` sights the rule *indicator* over observed-data minted from the detection's **own** fields, carrying the CAR guid only as an extension string — it never anchors to the spindle entity.

The north star wants the other shape: **emit the detection ↔ CAR-entity joins as STIX Sightings of the ATT&CK attack-patterns, over the spindle-identified observed-data — the behaviour timeline as the primary axis.**

What was built

A bridge — `dxdfir stix behaviour-sightings --car <car.db|tree> --detections <dir> --case <id>` — that:

1. **Parses the raw lane output.** suricata EVE `alert` events (technique from `alert.metadata.mitre_technique_id`), hayabusa timeline (technique from `MitreTags / OtherTags` , `ProcessGuid` from `Details.PGUID`), YARA memory/disk/file matches.
2. **Joins each to the CAR entity it touches**, against the finished `car.db` stores: suricata → the `flow` for the alert's **connection pair** (both endpoints), hayabusa → the `process` by `ProcessGuid` (case-folded, else `(host, pid)`), YARA → the `process` by `pid` or the `file` by `hash`.
3. **Emits one Sighting per (detection, entity, technique)** whose `sighting_of_ref` is the ATT&CK attack-pattern — MITRE's authoritative id (via the committed `attack_index`), referenced not shipped (BP §5.2) — over an `observed-data` **keyed on the matched CAR row's spindle guid** (so every detector that touches an entity references the *same* observation), with the host as `where_sighted_refs` . A flow observation resolves the C2 IP to its `dest_fqdn` as a `domain-name` SCO with `resolves_to_refs` — the alert read beyond its own source.

It reuses the exchange's object builders, bundle assembly and validation (`stix/objects.py` , `stix/export.py`), so a behaviour bundle merges object-for-object with the `stix export` and PIIAT bundles. Nothing is invented: a detection that names no resolvable technique, or joins no CAR entity, is counted and skipped — never given a fabricated attack-pattern or entity. Root pytest **373**, including 8 golden-vector tests for the bridge.

What we found on ls24-sample

Joined the three detection lanes (`data_store/processed/signatures/`) to the ls24 CAR stores (`zeek flow` , `evtx process` , `memory process`):

- **100 detections, every one joined a CAR entity** — 85 distinct entities flagged across the network, evtx and memory sources. The cross-source join is total: the pipeline names an entity for every alert.
- **1 Sighting of an ATT&CK attack-pattern:** hayabusa RecordID 5261 (`Cmd.EXE Missing Space Characters Execution Anomaly` , the DOSfuscated execution) → a **Sighting of attack-pattern-970a3432... (T1059.001)** over the evtx process's spindle observed-data (the `cmd.exe` file SCO, SHA-256), where sighted on **DESKTOP-M913391** . The bundle validates with **0 errors, 0 warnings** (ATT&CK ids resolve as the permitted non-local references).
- **99 joined, but no ATT&CK tag to sight.** The 14 suricata alerts all join the **specific C2 connection** `10.27.33.61 ↔ 100.101.0.42` — the flow that resolves to `scoring-c2.berylia.org` — but the ET Open rules that fired (`ET INFO SSH-2.0-Go...` , `ET SCAN ... Masscan`) carry no `mitre_technique_id` , so there is nothing to sight; the join is recorded, not invented into a technique. The 85 info-level hayabusa records and the YARA memory matches (DetectRaptor — e.g. a `DisableAntiSpyware` Defender-tamper indicator in `svchost`) likewise join a process without an ATT&CK tag.

The one attack-pattern sighting is the north star realised end-to-end on real data: a threat detection, lit up as a **behaviour** (T1059.001), anchored to the cross-source-resolved CAR entity rather than to the evtx record it came from.

What it enables

The bundle is the substrate for the **generative** half of the project. Each Sighting is a behaviour observed over the cascade; the shapes these sightings take — a technique over a spindle entity with its resolved network/host/memory context — are exactly what new CAR analytics should be **generated into**, codified against the broader mapped data sources rather than the ones MITRE's stock analytics assume.

Follow-ups

- **A technique-sparse corpus.** Only 1 of the 100 ls24 detections named a resolvable ATT&CK technique. Richer sightings need either ATT&CK-tagged rulesets (ET Open PRO / Sigma with `tags`) or the engine's own CAR analytics feeding the same bridge — both slot in without a shape change.
- **The lonewolf disk line-up.** This run is `ls24-sample` (network + evtx + memory). The single-host `lonewolf` disk line-up ([Step 06](#)) needs a **mount-enabled host** (`/dev/fuse`) so hayabusa image-EVTX extraction and yara-on-disk can run against its `car.db` ; the bridge itself is ready for it unchanged.
- **Joined-but-untagged detections.** The 99 joins are real cross-source links that today produce only a report count; emitting them as evidence (observed behaviour without an attack-pattern) is a candidate extension.